

Part 1: LMSR On-Chain AMM

1.1 What Is LMSR?

LMSR (Logarithmic Market Scoring Rule) is an automated market maker (AMM) specifically designed for **binary prediction markets**. It was introduced by Robin Hanson and formalized by Gnosis for the Conditional Tokens Framework.

Unlike Uniswap's CPMM (which prices two tokens against each other in a curve $x*y=k$), LMSR prices **outcome probabilities** for events with exactly two or more mutually exclusive outcomes.

The Formula

For a binary market (YES / NO):

$$P(\text{YES}) = \exp(q_{\text{yes}} / b) / (\exp(q_{\text{yes}} / b) + \exp(q_{\text{no}} / b))$$
$$P(\text{NO}) = \exp(q_{\text{no}} / b) / (\exp(q_{\text{yes}} / b) + \exp(q_{\text{no}} / b))$$

Where:

Variable	Meaning
q_yes	Net quantity of YES tokens the pool has issued/sold (liabilities)
q_no	Net quantity of NO tokens the pool has issued/sold (liabilities)
b	Liquidity parameter (funding). Higher b = flatter curve = less slippage
exp()	Exponential function ()
P(YES)	Marginal price of 1 YES token (0 to 1)

Key invariant: $P(\text{YES}) + P(\text{NO}) = 1$ at all times.

1.2 How the Pool Is Funded

When a Liquidity Provider (LP) seeds an LMSR pool, here is exactly what happens on-chain:

```
// 1. LP deposits 100 USDC (where b = 100 / ln(2) ≈ 144.26, but for simplicity let's use b=100)
factory.createLMSRMarketMaker(
  pmSystem,          // ConditionalTokens contract
  collateralToken,    // USDC
  conditionIds,       // market condition ID
  fee,                // e.g. 2% = 0.02 * 10^18
  whitelist,
  100 * 1e6           // 100 USDC as maximum funding
);
```

Initial Price

With zero tokens sold ($q_{\text{yes}} = 0$, $q_{\text{no}} = 0$): $P(\text{YES}) = \exp(0) / (\exp(0) + \exp(0)) = 1 / 2 = 0.5$

The initial price is always **0.5 USDC per token** when the pool is symmetrically funded. The parameter b doesn't change the starting price — it changes how much the price **moves** per trade.

1.3 Trading on LMSR — Step by Step

Example: Alice Buys 50 YES

State before trade:

- Tokens issued: $q_{\text{yes}} = 0$, $q_{\text{no}} = 0$
- $P(\text{YES}) = 0.5$

Alice calls:

```
pool.trade([50, 0], collateralLimit: 30);  
//      ^ YES  ^ NO      max USDC she's willing to pay
```

What `calcNetCost()` Does

The LMSR formula calculates the net cost deterministically based on the change in the pool's cost function $C(q)$:

```
// Cost Function:  $C(q) = b * \ln(\sum(\exp(q_i / b)))$   
// Cost of trade =  $C(q_{\text{after}}) - C(q_{\text{before}})$ 
```

For $b = 100$:

1. **Cost before trade:** $C_{\text{before}} = 100 * \ln(\exp(0) + \exp(0)) = 100 * \ln(2) \approx 69.31$ USDC
2. **Cost after trade (pool has now sold 50 YES):** $C_{\text{after}} = 100 * \ln(\exp(50/100) + \exp(0))$
 $C_{\text{after}} = 100 * \ln(1.6487 + 1) = 100 * \ln(2.6487) \approx 97.41$ USDC
3. **Net Cost:** $\text{Cost} = 97.41 - 69.31 = 28.10$ USDC

Alice pays ~28.10 USDC for 50 YES. (Average price of ~0.56 USDC/YES).

State after trade:

- Tokens issued: $q_{\text{yes}} = 50$, $q_{\text{no}} = 0$
- New Price: $P(\text{YES}) = \exp(50/100) / (\exp(50/100) + \exp(0)) = 1.6487 / 2.6487 = 0.622$

YES price rose from **0.50** → **0.622** because the pool has now issued more YES tokens. The LMSR formula interprets this as the market leaning toward YES.

Example: Alice Sells 20 YES (Taking Profits Early)

Users are **not** locked into their positions. Because the LMSR algorithm provides continuous mathematical liquidity, it acts as the "buyer of last resort." Alice can sell her tokens back to the pool *before* the market resolves to lock in her profits.

Alice calls:

```
pool.trade([-20, 0], collateralLimit: 12);  
//           ^ SELL YES           minimum USDC she expects back
```

Execution:

1. **Negative Cost Calculation:** The LMSR formula calculates the cost difference caused by decreasing the `q_yes` liability. Because `q_yes` is going down, the net cost is *negative* (e.g., `-12.44 USDC`), meaning the pool owes Alice money.
2. **Return Tokens:** Alice transfers her 20 YES tokens back to the pool.
3. **Payout:** The pool transfers 12.44 USDC from its own available collateral balance directly to Alice's wallet.
4. **Price Adjusts:** Because the pool took back YES tokens, the `q_yes` liability decreases. The LMSR formula interprets this as reduced demand, so the marginal price of YES drops slightly, mathematically rebalancing the market for the next trader.

Tuning the Liquidity Parameter () & Managing Risk

The parameter is the most critical variable in LMSR. It controls the "liquidity depth" of the market, which directly dictates the **maximum bounded loss** for the Liquidity Provider (LP).

Because the LP (the platform) must subsidize the early liquidity, the absolute maximum amount the LP can ever lose on a single market is mathematically capped at ****. (Once the price reaches 0.999 USDC, users are paying 1 USDC to win 1 USDC, so the LP stops losing money).

- **High (e.g., 10,000):**
 - **User Experience:** Deep liquidity and low slippage. It requires massive trading volume to move the price, which is excellent for large traders.
 - **Platform Risk:** High maximum bounded loss (tokens). The platform locks up significant capital. Best for highly anticipated markets where trading fees will offset the risk.
- **Low (e.g., 100):**
 - **User Experience:** Shallow liquidity and high slippage. A small trade will cause a massive price swing, creating a poor user experience.
 - **Platform Risk:** Very low maximum bounded loss (tokens) for the LP. Best for low-traffic ghost-town markets where you don't want to risk much capital.

Part 2: Official Polymarket Order Matching (CTFExchange)

2.1 Architecture Overview

Polymarket's **CTFExchange** is a completely different approach from LMSR. Instead of an algorithmic AMM, it uses an **off-chain orderbook with on-chain settlement**.

(Diagram omitted for brevity - same architecture as original)

2.2 The Order Struct

Every trade starts with a signed order. Here is the official Polymarket order struct (modern V2):

```

struct Order {
    uint256 salt;           // Unique salt for replay protection
    address maker;          // Address placing the order
    address signer;        // Signer (may differ from maker for vaults)
    uint256 tokenId;       // ERC-1155 token ID (YES or NO position)
    uint256 makerAmount;   // Amount of asset maker sells (or collateral for BUY)
    uint256 takerAmount;   // Amount of asset maker wants to buy
    Side side;             // BUY or SELL
    SignatureType signatureType; // EOA, EIP-1271, or Gnosis Safe
    uint256 nonce;         // Order nonce for batch cancellation
    uint256 timestamp;     // Order validity start time
    bytes32 metadata;      // Additional data (e.g. fee tier reference)
    bytes32 builder;       // Builder address for fee sharing
}

```

(Note: The signature itself is passed externally, not inside the struct that gets hashed).

EIP-712 TypeHash

```

bytes32 constant ORDER_TYPEHASH = keccak256(
    "Order(uint256 salt,address maker,address signer,uint256 tokenId,"
    "uint256 makerAmount,uint256 takerAmount,uint8 side,uint8 signatureType,"
    "uint256 nonce,uint256 timestamp,bytes32 metadata,bytes32 builder)"
);

```

2.3 Order Storage & Cancellation

Orders are **never stored on-chain**. The only on-chain tracking is:

```

mapping(bytes32 => OrderStatus) public orderStatus;

```

Nonces are On-Chain (for Cancellation): Because the `nonce` is included in the signed order, a user can invalidate all previously signed orders (or a subset) by incrementing their nonce on-chain:

```

function incrementNonce(uint256 val) external {
    nonces[msg.sender] += val;
}

```

2.4 Match Types — Three Scenarios

1. COMPLEMENTARY (Buy vs Sell)

- **Scenario:** Alice is BUYING YES, Bob is SELLING YES.
- **Execution:** A direct swap of tokens (ERC-1155 YES for USDC) between Alice and Bob. No interaction with ConditionalTokens.

2. MINT (Buy vs Buy)

- **Scenario:** Alice wants to BUY 100 YES, Charlie wants to BUY 100 NO. Both pay 50 USDC.

- **Execution:**
 - Exchange collects 100 USDC total.
 - Calls `ctf.splitPosition` which takes 100 USDC and **mints 100 YES + 100 NO**.
 - Transfers 100 YES to Alice, 100 NO to Charlie.

3. MERGE (Sell vs Sell)

- **Scenario:** Alice wants to SELL 100 YES, Charlie wants to SELL 100 NO. Both receive 50 USDC.
- **Execution:**
 - Exchange collects 100 YES and 100 NO.
 - Calls `ctf.mergePositions` which burns the 100 YES + 100 NO and **unlocks 100 USDC from the CTF contract escrow**.
 - Transfers 50 USDC to Alice, 50 USDC to Charlie.

2.5 Security Model

What Can Go Wrong and How It's Prevented

Attack	How CTFExchange Prevents It
Replay attack	<code>orderStatus[orderHash].filled</code> prevents the same order being matched twice
Signature malleability	EIP-712 standard signature verification
Operator spread capture	The operator can't change the limit price, but can match orders with their own liquidity to capture the spread (implicit front-running). This is a known limitation of off-chain matchers.
Fee theft	Fee amounts are capped and verified on-chain